S-INFO-444: QUIC Project

Quentin De Coninck

March 24, 2025

1 Context

As described in the lectures, QUIC is a protocol which enables connections to survive IP address changes over time. It is particularly useful in the context of NAT'ed clients, where the NAT may rebind the public source port allocated to some client. This operation is called "NAT rebinding" and is one of the main motivations for the connection migration feature of QUIC. While such changes are visible at server side, clients are agnostic to such events. To address this, STUN[1] is a protocol enabling communicating the observed peer address. Still, this is a side protocol running most of the time directly on UDP or TCP.

For this project, you will implement a QUIC extension enabling QUIC endpoints to communicate seen IP addresses and ports from their peer. This extension will be implemented in the Rust `quiche` implementation, reachable from `https://github.com/cloudflare/quiche`. For simplicity purposes, please start your project on commit `c6489c8`.

2 Extension Specification

The QUIC address discovery specification is available at `https://datatracker.ietf.org/doc/draft-ietf-quic-address-discovery/00/`. Note that this project focuses on `draft-ietf-quic-address-discovery-00`. In the case the document evolves over time (e.g., goes in version 01), the project must implement version 00.

3 Deliverables

This project is done by groups of at most two students. Your implementation of the extension **MUST** be interoperable with others, i.e., you cannot define frames/transport parameters beyond the ones included in the specification. Note, however, that the specification gives some freedom to the behavior of your implementation[2]. Furthermore, you are free to add dependencies if they are relevant for your implementation.

The extension needs to be implemented in the core `quiche` crate. You will also need to extend the `quiche-client` and `quiche-server` example applications to enable the use of your extension[3].

[1] `https://datatracker.ietf.org/doc/rfc8489/`.

[2] This is intentional, and is therefore prone to discussion.

[3] You are free to define additional specific applications.

To ensure that your implementation is compliant to the specification, we will organize **interoperability test sessions** during the last two practical sessions on **May 5th and May 12th**. Note that while taking part to these sessions is not mandatory, we expect you to perform interoperability tests with **at least two other groups**[4].

The deadline of the project is set on **May 16 th, 2025 at 20h**. Your submission will consist in the following elements.

- A PDF report containing at least the following elements:
  - The different implementation choices you took.
  - How you assess that your implementation "works".
  - How to run your `quiche-client` and `quiche-server` with the extension enabled. You may also create `async`-based applications by building on the `tokio-quiche` crate for better performance (bonus point if you can explain why).
  - The potential attacks on the extension taken into account, and their defense put in place, **with concrete, evaluated examples**.
  - A **report describing your interoperability tests** with implementations from different groups: what have you done (actual performed tests), what have you discovered,...
  - How you would design a network attacker that would break this extension.
- Your `git` repository of your modified `quiche` implementation; either as a .ZIP file[5] or as a link to your **private** GitHub/GitLab/Bitbucket repository where access has been given to the teacher. The modified `quiche` should at least contain
  - The implementation of the specified extension;
  - (Unit) Tests showing that your extension's implementation works *in normal conditions*[6];
  - (Unit) Tests demonstrating possible attacks on the extension, and how your implementation handles them.

4 Advices

Don't `panic!()`; although the complete `quiche` implementation consists in more than 65000 lines of Rust code[7] (whose the core `quiche` crate is close to 44000 lines alone), you should not have too much troubles to find out where you should apply your modifications. The hard part is having a first working prototype. To reach it, we advise you to follow this pattern.

1. Spend some time performing the practical session dedicated to QUIC. You will start playing with the `quiche` implementation as a user – which is useful if you need to understand which API the implementation provides.

[4] Of course, you can experiment with more groups.

[5] With this option, please run `cargo clean` and remove the `boringssl` submodule before creating the archive.

[6] It is up to you to define what is meant by *normal conditions*.

[7] Note that the root `quiche` folder is a workspace having multiple crates, such as `quiche`, `octets`, `tokio-quiche` and `apps`.

2. Look at both the `quiche-client.rs` and `quiche-server.rs` examples in `/apps` and find how you could *configure* your QUIC connection to *enable*[8] the *QUIC address discovery* extension.

3. From now on, most of your focus should be on the core `quiche` crate. Once a connection is configured to negotiate the extension, implement the extension transport parameter.

4. Once the extension has been successfully negotiated, integrate the new frames. Note how frames are implemented in `quiche`; can you find all the places where some modification should be performed?

5. Perform iterative development: declare functions with `todo!()` placeholders, write a test function to assert the expected behavior, then implement the block making the test failing, and repeat until the test finally passes.

6. Adding the new transport parameter and frames in the QLOG format is quite straightforward, and may be useful for debugging purposes.

[8] Note that there are several modes to the extension.